

OceanEmbed-NIO Architecture & Modification Guide

This document explains exactly how the OceanEmbed-NIO project is structured, where critical files are stored, and how you can directly modify the model, data, and user interface so that your live deployment updates seamlessly.

1. The Frontend (Level 1 Million UI)

The entire user interface, including the 3D interactive maps, cinematic hero sections, and official government branding, is completely decoupled from the AI model.

Where it is stored:

- * `/home/pseudo/ML/SIH2026/version4/oceanembed-kit/web/index.html`: The main HTML structure (Govt headers, text, layout).
- * `/home/pseudo/ML/SIH2026/version4/oceanembed-kit/web/incois.css`: The ultra-premium styling, glassmorphism, colors, and animations.
- * `/home/pseudo/ML/SIH2026/version4/oceanembed-kit/web/app.js`: The Javascript logic that fetches data from the backend, generates the 3D Plotly graphs, and handles camera animations.

How to change it:

If you want to edit the text on the website, change colors, or modify the 3D camera angles, you simply edit these three files. Because the backend serves them statically, **changes made here are instantly reflected on your live link without needing to restart the server.**

2. The AI Model Weights & Logic

The actual intelligence of the system—the deep CNN-Attention network, the Argo observer calibration, and conformal prediction—is stored in PyTorch checkpoint files.

Where it is stored:

- * `/home/pseudo/ML/SIH2026/version4/oceanembed-kit/runs/core/checkpoint.pt`: The primary deep learning backbone weights.
- * `/home/pseudo/ML/SIH2026/version4/oceanembed-kit/runs/observer/checkpoint.pt`: The Argo calibration weights.
- * `/home/pseudo/ML/SIH2026/version4/oceanembed-kit/runs/conformal.json`: The statistical uncertainty calibration mappings.

How to change it:

If you improve the neural network architecture in `oceanembed/architecture.py`, you must retrain the model. Running `python3 -m oceanembed.train --pilot` will overwrite these `.pt` files with your new, smarter weights.

3. The Live Data (Predictions & Output)

The website does not run the heavy AI model in real-time when a user visits. Instead, it reads pre-computed daily predictions (inference).

Where it is stored:

- * `/home/pseudo/ML/SIH2026/version4/oceanembed-kit/outputs/public/`: This is the most critical folder for the live site.
- * Inside this folder, there are compressed NumPy arrays (e.g., `20200630.npz`). These contain the actual 3D grids of temperature and anomalies that the website visualizes.
- * `outputs/public/manifest.json`: Tells the website which dates are available to show in the dropdown menus.

How to change it:

When you want to add new days of data or push new model predictions to the live website, you run:

`python3 -m oceanembed.run --pilot` (or without `--pilot` for the full dataset).

This command automatically feeds new satellite data through your trained model and writes fresh `.npz` files into `outputs/public/`. The live website will automatically detect them and allow users to view the new data!

4. The API Server

The bridge between the `.npz` data files and the web frontend is a high-speed Python FastAPI server.

Where it is stored:

- * `/home/pseudo/ML/SIH2026/version4/oceanembed-kit/oceanembed/api.py`

How it works:

The server is currently running in the background via `uvicorn`. It reads the arrays in `outputs/public/` and slices them into lightweight JSON that `app.js` can render. If you need to add new variables (like salinity), you would add a new endpoint here.

Summary Workflow: "How do I upgrade the live link?"

1. **To change the look:** Edit `web/index.html` or `web/incois.css`. (Updates instantly).
2. **To improve the AI:** Edit the python files, run `python3 -m oceanembed.train`, then run `python3 -m oceanembed.run`. The new `.npz` files will overwrite the old ones, and the live link will immediately show the new, more accurate predictions!